[Improving LLM Classification of Logical Errors by Integrating Error Relationship into Prompts]

- **Problem**: Logical errors do not produce compiler error messages, making them difficult for novice programmers to identify.
- **Proposed Solution**: A classification system of ten logical error types and a prompt engineering approach using Chain-of-Thought (CoT) and Tree-of-Thought (ToT).
- **Key Finding**: Providing error descriptions in prompts increased average classification accuracy by 21%.

Classification of Logical Errors

The authors defined ten error types to provide clear criteria for LLM classification.

ID	Category	Description / Example
(A)	**Input**	Failing to receive all input values or using the incorrect data type for variables.
(B)	**Output**	Non-compliance with required formats or outputting incorrect string literals.
(C)	**Variable**	Storing incorrect values or specifying the wrong data type for a variable.
(D)	**Computation**	Calculating with incorrect values or using the wrong operations.
(E)	**Condition**	Incorrect or insufficient conditional operations in declarations.
(F)	**Branching**	Incorrect program flow (e.g., wrong break placement or if-if instead of if-else).
(G)	**Loop**	Incorrect conditions or variables used in loop declarations.
(H)	**Array/String**	Incorrect initialization or referencing an incorrect index.
(I)	**Function**	Incorrectly defined parameters/return values or wrong arguments in calls.
(J)	**Conceptual**	Solving the wrong problem or missing necessary loops/conditions entirely.

Higher Level Category	Included Error Types	Explanation
(I) Problem Understanding and Algorithm Design	(J)	Errors caused by misunderstanding the problem or designing an incorrect or incomplete overall solution approach.
(B) Interface and Contract	(A), (B), (I)	Errors arising from violations of input, output, or function interface specifications and data exchange contracts.
(C) State Representation	(C), (H)	Errors related to incorrect storage, initialization, or referencing of program data and structures.
(D) Control Logic	(E), (F), (G)	Errors that disrupt the intended execution flow through incorrect conditions, branches, or loops.
(E) Computation and Expression	(D)	Errors involving incorrect calculations, operators, or expressions applied to otherwise valid data.

Error Relationships and Priority

The study identifies two types of relationships that complicate error detection:

- **Casual Relationships**: Errors in early program steps (Input) leading to subsequent errors in later steps (Process/Output).
- **Coincidence Relationships**: Overlapping definitions, such as a "Computation" error occurring within a "Condition" declaration.

To resolve ambiguity, the authors established a priority ordering for classification:

(J) > (I) > (A) > (C) > (H) > (E) = (G) > (D) > (F) > (B)

Relationship of Prioritized Groups

Priority Level	Error Category	Logic for Placement
Highest	**(J) Conceptual**	Overlaps with all other concepts; represents the broadest misunderstanding.

High	**(I) Function, (A) Input**	Core components or early-stage IPO steps that trigger later errors.
Middle	**(C) Variable, (H) Array, (E) Condition, (G) Loop**	"Process" errors with specific code locations that frequently overlap with others.
Lowest	**(D) Comp., (F) Branching, (B) Output**	Often the result of earlier errors or occur in ambiguous code areas.

Results

Classification Using LLMs

4.1 Logical Error Classification Prompt

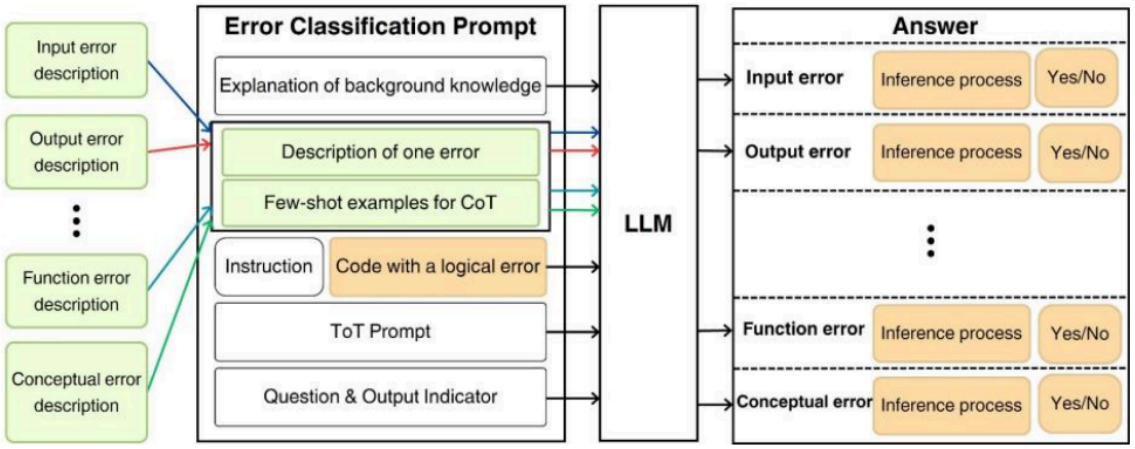

Figure 3. This is the overall structure and process for classifying logical error types from incorrect answer codes.

Classification Performance (Accuracy)

	GPT-3.5-Turbo			GPT-4	
	Description not provided	Description provided		Description provided	
Error Type	**Acc**	**Acc**	**FPR**	**Acc**	**FPR**
Input	20% (2/10))	50% (5/10)	0.06	100% (10/10)	0.1
Output	20% (2/10)	80% (8/10)	0.1	100% (10/10)	0.08
Variable	40% (2/5)	60% (3/5)	0.22	100% (5/5)	0.12
Computation	67% (6/9)	44% (4/9)	0.12	55% (5/9)	0.12
Condition	50% (6/12)	67% (8/12)	0.25	92% (11/12)	0.125
Branching	38% (3/8)	50% (4/8)	0.188	63% (5/8)	0.188
Loop	50% (5/10)	50% (5/10)	0.18	100% (10/10)	0.1
Array/String	25% (2/8)	50% (4/8)	0.138	88% (7/8)	0.275
Function	25% (2/8)	63% (5/8)	0.038	88% (7/8)	0.075
Conceptual	0% (0/6)	33% (2/6)	0.167	67% (4/6)	0.15
AVG	**35% (30/86)**	**56% (48/86)**	**0.145**	**86% (74/86)**	**0.13**